

assignment1.out

January 20, 2026

```
# install
!pip install numpy
!pip install matplotlib
!pip install pandas
!pip install openpyxl
!pip install scipy
!pip install nbconvert
!pip install pandoc
```

Requirement already satisfied: numpy in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (2.4.1)

Requirement already satisfied: matplotlib in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (3.10.8)

Requirement already satisfied: contourpy>=1.0.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (4.61.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (1.4.9)

Requirement already satisfied: numpy>=1.23 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (2.4.1)

Requirement already satisfied: packaging>=20.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (25.0)

Requirement already satisfied: pillow>=8 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (12.1.0)

Requirement already satisfied: pyparsing>=3 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (3.3.1)

Requirement already satisfied: python-dateutil>=2.7 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: pandas in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (2.3.3)

Requirement already satisfied: numpy>=1.26.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandas) (2.4.1)

Requirement already satisfied: python-dateutil>=2.8.2 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandas) (2025.3)

Requirement already satisfied: six>=1.5 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: openpyxl in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (3.1.5)

Requirement already satisfied: et-xmlfile in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from openpyxl) (2.0.0)

Requirement already satisfied: scipy in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (1.17.0)

Requirement already satisfied: numpy<2.7,>=1.26.4 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from scipy) (2.4.1)

Requirement already satisfied: nbconvert in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (7.16.6)

Requirement already satisfied: beautifulsoup4 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (4.14.3)

Requirement already satisfied: bleach!=5.0.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0->nbconvert) (6.3.0)

Requirement already satisfied: defusedxml in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.7.1)

Requirement already satisfied: Jinja2>=3.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.1.6)

Requirement already satisfied: jupyter-core>=4.7 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.9.1)

Requirement already satisfied: jupyterlab-pygments in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.3.0)

Requirement already satisfied: markupsafe>=2.0 in

/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.0.3)
 Requirement already satisfied: mistune<4,>=2.0.3 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.2.0)
 Requirement already satisfied: nbclient>=0.5.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.10.4)
 Requirement already satisfied: nbformat>=5.7 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.10.4)
 Requirement already satisfied: packaging in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (25.0)
 Requirement already satisfied: pandocfilters>=1.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (1.5.1)
 Requirement already satisfied: pygments>=2.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (2.19.2)
 Requirement already satisfied: traitlets>=5.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.14.3)
 Requirement already satisfied: webencodings in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
 Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
 Requirement already satisfied: platformdirs>=2.5 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-core>=4.7->nbconvert) (4.5.1)
 Requirement already satisfied: jupyter-client>=6.1.12 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbclient>=0.5.0->nbconvert) (8.8.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
 Requirement already satisfied: pyzmq>=25.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (27.1.0)
 Requirement already satisfied: tornado>=6.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.4)
 Requirement already satisfied: fastjsonschema>=2.15 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbformat>=5.7->nbconvert) (2.21.2)
 Requirement already satisfied: jsonschema>=2.6 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-

packages (from nbformat>=5.7->nbconvert) (4.26.0)
 Requirement already satisfied: attrs>=22.2.0 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.1)
 Requirement already satisfied: referencing>=0.28.4 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
 Requirement already satisfied: rpds-py>=0.25.0 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
 Requirement already satisfied: six>=1.5 in /Users/aeshagandhi/Downloads/MIDS-
 Sp26/IDS-705/.venv/lib/python3.12/site-packages (from python-
 dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
 Requirement already satisfied: typing-extensions>=4.4.0 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from referencing>=0.28.4->jsonschema>=2.6->nbformat>=5.7->nbconvert)
 (4.15.0)
 Requirement already satisfied: soupsieve>=1.6.1 in
 /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
 packages (from beautifulsoup4->nbconvert) (2.8.3)
 Requirement already satisfied: pandoc in /Users/aeshagandhi/Downloads/MIDS-
 Sp26/IDS-705/.venv/lib/python3.12/site-packages (2.4)
 Requirement already satisfied: plumbum in /Users/aeshagandhi/Downloads/MIDS-
 Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandoc) (1.10.0)
 Requirement already satisfied: ply in /Users/aeshagandhi/Downloads/MIDS-
 Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandoc) (3.11)

0.0.1 Exercise 1

0.0.2 1.1

$P(X = 10 | \text{Roll } 1 = 1) = P(\text{Roll } 2 + 3 = 9)$

Roll 2 and 3 sum to 9 if the outcomes of roll 1 and 2 are (3,6), (4,5), (5,4), (6,3) out of 36 possible outcomes

Thus 4 out of 36 possible outcomes $\rightarrow 1/9$

0.0.3 1.2

```

import random
dice = [1,2,3,4,5,6]
roll = random.choice(dice)

counter = 0
for i in range(1000000):

```

```

roll_1 = random.choice(dice)
roll_2 = random.choice(dice)
roll_3 = random.choice(dice)
if roll_1 == 1:
    total = roll_1 + roll_2 + roll_3
    if total == 10:
        counter += 1

print("The probability of rolling a total of 10 with the first die being 1 is_
↪approximately:", counter / 1000000)

```

The probability of rolling a total of 10 with the first die being 1 is approximately: 0.01861

0.0.4 1.3

$$P(\text{positive} \mid \text{has disease}) = 0.95$$

$$P(\text{positive} \mid \text{no disease}) = 0.03$$

$$P(\text{has disease}) = 1/1000$$

$$P(\text{has disease} \mid \text{positive}) = (P(\text{positive} \mid \text{has disease}) * P(\text{has disease})) / (P(\text{positive}))$$

$$P(\text{positive}) = P(\text{has disease}) P(\text{positive} \mid \text{has disease}) + P(\text{no disease}) P(\text{positive} \mid \text{no disease})$$

$$\text{Then, } P(\text{has disease} \mid \text{positive}) = (0.001 * 0.95) / (0.001 * 0.95 + (1-0.001) * (0.03))$$

$$P(\text{has disease} \mid \text{positive}) = 0.0307$$

```

q1_3 = (0.001 * 0.95) / (0.001*0.95 + (1-0.001)*(0.03))
print("The probability that a person has the disease given that they tested_
↪positive is approximately:", q1_3)

```

The probability that a person has the disease given that they tested positive is approximately: 0.03072445019404916

0.0.5 1.4

$$E[X] = -1 * P(X = -1) + 0 * P(X = 0) + 1 * P(X = 0.3)$$

$$= -1 * 0.2 + 1 * 0.3$$

$$= 0.1$$

$$\text{Var}[X] = E[X^2] - E[X]^2$$

$$E[X^2] = (-1)^2 * P(X = -1) + 0^2 * P(X = 0) + 1^2 * P(X = 0.3)$$

$$= 1 * P(X = -1) + 1 * P(X = 0.3)$$

$$= 1 * 0.2 + 1 * 0.3$$

$$= 0.5$$

Then, $\text{Var}[X] = E[X^2] - E[X]^2$

$$= (0.5)^2 - (0.09999)^2$$

$$= 0.24$$

See code below for full output:

```
theoretical_mean = (-1 * 0.2) + (1 * 0.3)
# rounding to avoid floating point precision issues

theoretical_var = ((1 * 0.2 + 1 * 0.3) ** 2) - (theoretical_mean**2)
theoretical_std = theoretical_var ** 0.5

print("Expected value E[X]:", round(theoretical_mean, 2))
print("Variance Var(X):", round(theoretical_var, 2))
print("Standard deviation:", round(theoretical_std, 2))
```

Expected value E[X]: 0.1

Variance Var(X): 0.24

Standard deviation: 0.49

0.1 Exercise 2

0.1.1 2.1

```
# load in wait_times.csv using np.loadtxt()
import numpy as np
wait_times = np.loadtxt('HW1/wait_times.csv', delimiter=',')

# plot histogram of wait_times

import matplotlib.pyplot as plt
plt.hist(wait_times, bins=10, range=(0, 1))
plt.xlabel('Wait Time (as fraction of 8 hour work day)')
plt.ylabel('Frequency')
plt.title('Histogram of Wait Times')
plt.show()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[8], line 3
      1 # load in wait_times.csv using np.loadtxt()
      2 import numpy as np
----> 3 wait_times = np.loadtxt(                , delimiter= )
```

```

5 # plot histogram of wait_times
7 import matplotlib.pyplot as plt

```

```

File ~/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages/numpy/lib
↳_npio_impl.py:1384, in loadtxt(fname, dtype, comments, delimiter, converters,
↳skiprows, usecols, unpack, ndmin, encoding, max_rows, quotechar, like)
    1381 if isinstance(delimiter, bytes):
    1382     delimiter = delimiter.decode('latin1')
-> 1384 arr = _read(fname, dtype=dtype, comment=comment, delimiter=delimiter,
    1385               converters=converters, skiplines=skiprows, usecols=usecols,
    1386               unpack=unpack, ndmin=ndmin, encoding=encoding,
    1387               max_rows=max_rows, quote=quotechar)
    1389 return arr

```

```

File ~/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages/numpy/lib
↳_npio_impl.py:1011, in _read(fname, delimiter, comment, quote,
↳imaginary_unit, usecols, skiplines, max_rows, converters, ndmin, unpack,
↳dtype, encoding)
    1009     fname = os.fspath(fname)
    1010 if isinstance(fname, str):
-> 1011     fh = np.lib._datasource.open(fname, encoding=encoding)
    1012     if encoding is None:
    1013         encoding = getattr(fh, 'encoding', 'latin1')

```

```

File ~/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages/numpy/lib
↳_datasource.py:192, in open(path, mode, destpath, encoding, newline)
    155 """
    156 Open `path` with `mode` and return the file object.
    157
    (...)    188
    189 """
    191 ds = DataSource(destpath)
--> 192 return ds.open(path, mode, encoding=encoding, newline=newline)

```

```

File ~/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages/numpy/lib
↳_datasource.py:529, in DataSource.open(self, path, mode, encoding, newline)
    526     return _file_openers[ext](found, mode=mode,
    527                               encoding=encoding, newline=newline)
    528 else:
--> 529     raise FileNotFoundError(f"{path} not found.")

```

FileNotFoundError: HW1/wait_times.csv not found.

0.1.2 2.2

```
# mean, variance, std of wait times
# convert mean and std to hours
mean_wait = np.mean(wait_times)
mean_wait_hours = mean_wait * 8 # convert to hours
var_wait = np.var(wait_times)
std_wait = np.std(wait_times)
std_wait_hours = std_wait * 8 # convert to hours

# print and describe in both original units and converted units
print(f"Mean wait time: {mean_wait:.4f} (fraction of work day), \u2192 {mean_wait_hours:.2f} hours")
print(f"Variance of wait times: {var_wait:.6f} (fraction of work day squared)")
print(f"Standard deviation of wait times: {std_wait:.4f} (fraction of work \u2192 day), {std_wait_hours:.2f} hours")
```

Mean wait time: 0.4042 (fraction of work day), 3.23 hours
Variance of wait times: 0.081548 (fraction of work day squared)
Standard deviation of wait times: 0.2856 (fraction of work day), 2.28 hours

0.1.3 2.3

To form a valid probability distribution, it must sum to 1 over all possible values.

$$\int_0^1 \alpha e^{-x} dx = 1$$

$$\alpha \int_0^1 e^{-x} dx = 1$$

$$\alpha [-e^{-x}]_0^1 = 1$$

$$\alpha (-e^{-1} + e^0) = 1$$

$$\alpha \left(1 - \frac{1}{e}\right) = 1$$

$$\alpha = \frac{1}{\left(1 - \frac{1}{e}\right)} = \frac{e}{e-1}$$

The value of alpha that makes the pdf a valid probability density function is $\frac{e}{e-1} \sim 1.582$.

```
alpha = np.e / (np.e - 1)
alpha
```


[]: 1.5819767068693265

0.1.4 2.4

2.4/ compute cdf

$$\int_0^x \alpha e^{-x} dx = \alpha \int_0^x e^{-x} dx = \alpha \left[-e^{-x} \right]_0^x = \alpha (1 - e^{-x})$$

then,

$$F_X(X) = \begin{cases} 0 & x < 0 \\ \alpha(1 - e^{-x}) & 0 \leq x \leq 1 \\ 1 & x > 1 \end{cases}$$

where $\alpha = \frac{e}{e-1} \sim 1.582$

0.1.5 2.5

$$E[X] = \int_{-\infty}^{\infty} x f_X(x) dx = \int_0^1 x \alpha e^{-x} dx = \alpha \int_0^1 x e^{-x} dx$$

use integration by parts

$$\begin{array}{ll} f = x & g' = e^{-x} \\ f' = 1 & g = -e^{-x} \end{array}$$
$$-xe^{-x} - \int -e^{-x} dx \Rightarrow -xe^{-x} - e^{-x}$$
$$\Rightarrow \alpha \left[-xe^{-x} - e^{-x} \right]_0^1 = \alpha \left[-(x+1)e^{-x} \right]_0^1 = \alpha \left(1 - \frac{2}{e} \right)$$

plug in $\alpha \rightarrow \frac{e}{e-1} \left(1 - \frac{2}{e} \right) = \frac{e}{e-1} - \frac{2}{e-1} = \frac{e-2}{e-1}$

$$E[X] = \alpha \left(1 - \frac{2}{e} \right) = \frac{e-2}{e-1} \approx 0.418$$

Then, # hours waiting is $\approx 8(0.418) = 3.344$ hours

0.1.6 2.6

$$\begin{aligned} \text{var}(X) &= E(X^2) - E(X)^2 \\ E(X^2) &= \int_0^1 x^2 \cdot \alpha e^{-x} dx = \alpha \int_0^1 x^2 e^{-x} dx \end{aligned}$$

$f = x^2 \quad g' = e^{-x}$
 $f' = 2x \quad g = -e^{-x}$

$$\begin{aligned} &= \alpha \left[-x^2 e^{-x} - \int -2x e^{-x} dx \right] \quad \text{use from } E(X) \text{ calculation} \\ &= \alpha \left[-x^2 e^{-x} + 2 \int (1 - \frac{x}{e}) \right] \\ &= \alpha \left[-\frac{1}{e} + 2 - \frac{4}{e} \right] \\ &= \alpha \left(2 - \frac{5}{e} \right) \quad \text{plug in } \alpha \\ &= \frac{e}{e-1} \left(2 - \frac{5}{e} \right) = \frac{2e}{e-1} - \frac{5}{e-1} = \frac{2e-5}{e-1} \end{aligned}$$

$$\text{var}(X) = \overbrace{\alpha \left(2 - \frac{5}{e} \right)}^{E(X^2)} - \left(\overbrace{\alpha \left(1 - \frac{2}{e} \right)}^{E(X)} \right)^2 = \frac{2e-5}{e-1} - \left(\frac{e-2}{e-1} \right)^2 \approx 0.0793$$

then, $\text{SD}(X) \approx 0.282 \Rightarrow$ in hours, $8(0.282) = 2.256$ hours

0.1.7 2.7

```
alpha = np.e/(np.e - 1)

def pdf(x):
    if 0 <= x <= 1:
        return alpha * np.exp(-x)
    else:
        return 0

def cdf(x):
    if x < 0:
        return 0
    elif 0 <= x <= 1:
        return alpha * (1 - np.exp(-x))
    else:
        return 1

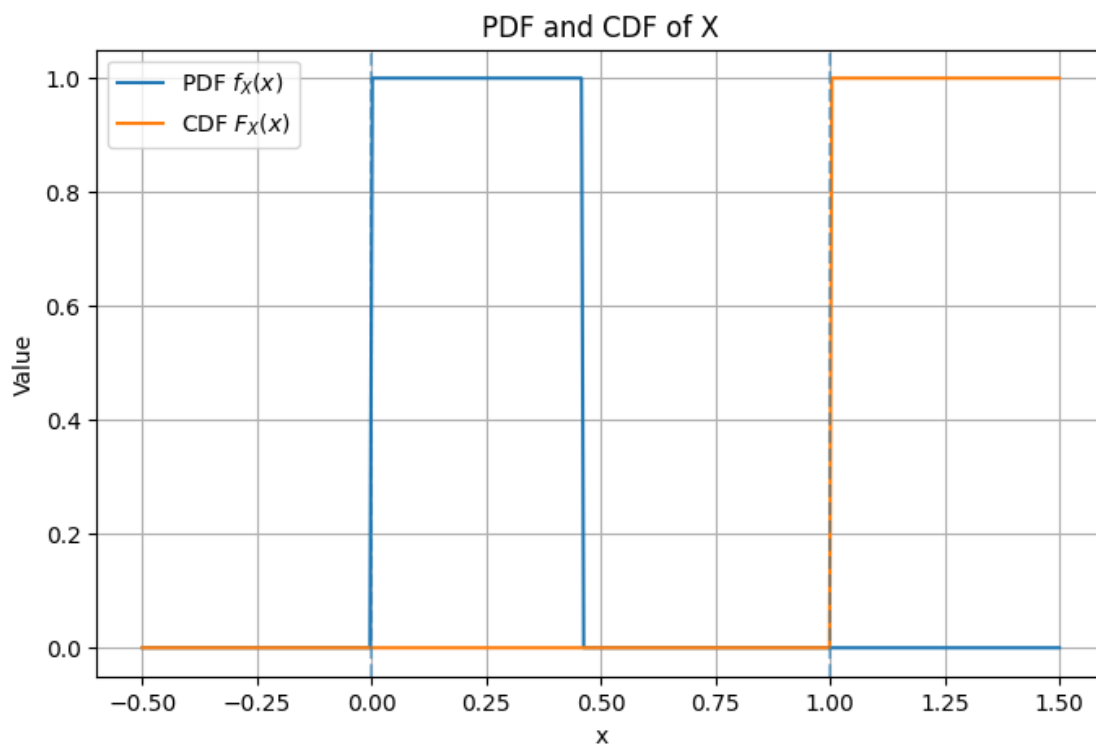
v_pdf = np.vectorize(pdf)
v_cdf = np.vectorize(cdf)

x_vals = np.linspace(-0.5, 1.5, 400)
pdf_vals = v_pdf(x_vals)
cdf_vals = v_cdf(x_vals)
```

```

plt.figure(figsize=(8,5))
plt.plot(x_vals, pdf_vals, label="PDF  $f_X(x)$ ")
plt.plot(x_vals, cdf_vals, label="CDF  $F_X(x)$ ")
plt.axvline(0, linestyle="--", alpha=0.5)
plt.axvline(1, linestyle="--", alpha=0.5)
plt.xlabel("x")
plt.ylabel("Value")
plt.title("PDF and CDF of X")
plt.legend()
plt.grid(True)
plt.show()

```



2.8

```

# empirical cdf from wait times data by sorting data in ascending order of  $x$ 
# values, and assigning y as cumulative probabilities
sorted_wait_times = np.sort(wait_times)
n = len(sorted_wait_times)
empirical_cdf = np.arange(1, n+1) / n

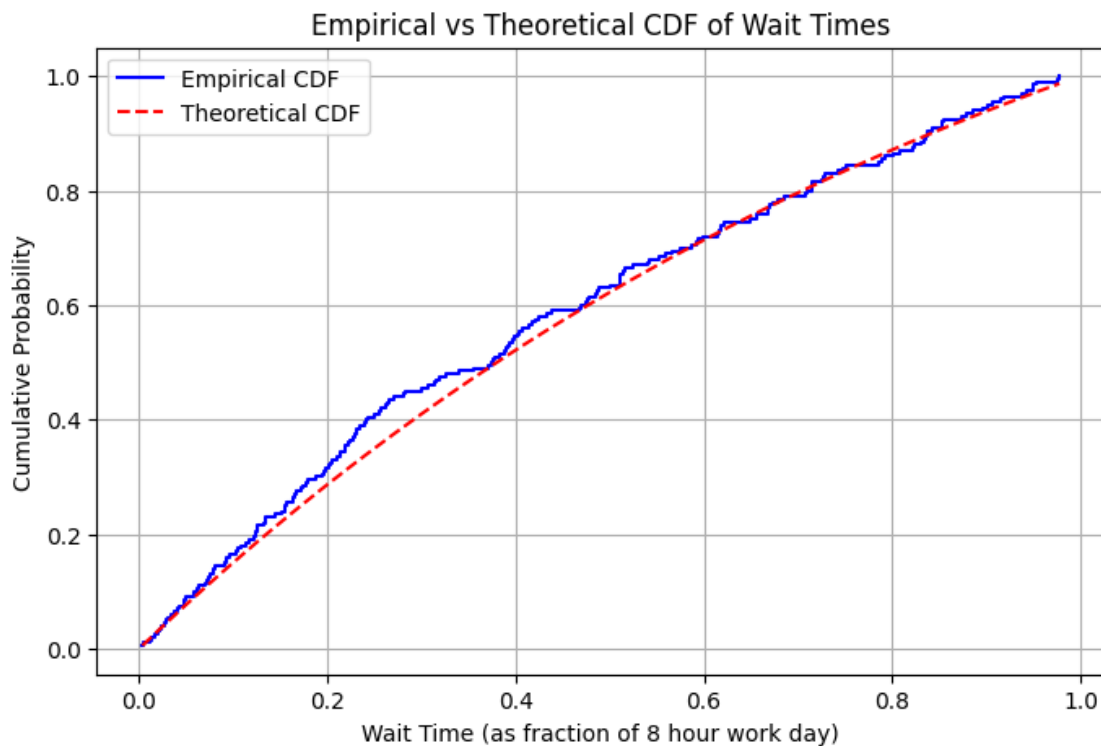
# plot empirical cdf and compare to the theoretical cdf
theoretical_cdf_vals = v_cdf(sorted_wait_times)

```

```

plt.figure(figsize=(8,5))
plt.step(sorted_wait_times, empirical_cdf, where='post', label='Empirical CDF',
        color='blue')
plt.plot(sorted_wait_times, theoretical_cdf_vals, label='Theoretical CDF',
        color='red', linestyle='--')
plt.xlabel('Wait Time (as fraction of 8 hour work day)')
plt.ylabel('Cumulative Probability')
plt.title('Empirical vs Theoretical CDF of Wait Times')
plt.legend()
plt.grid(True)
plt.show()

```



2.9

```

# inverse cdf
"""
Create a numerical simulation of this process. Doing this for a custom PDF is
easier than you may think. We typically have access to uniformly distributed
samples (through np.random.rand), and we can transform these uniform samples
into any distribution we wish. To do this, we can input uniform variates
through the inverse of the CDF
"""
# already have cdf defined above

```

```

def inverse_cdf(u):
    if 0 <= u < alpha * (1 - np.exp(-1)):
        return -np.log(1 - u / alpha)
    elif u >= alpha * (1 - np.exp(-1)) and u <= 1:
        return 1
    else:
        raise ValueError("u must be in [0, 1]")
v_inverse_cdf = np.vectorize(inverse_cdf)
# generate uniform samples and transform using inverse cdf
uniform_samples = np.random.rand(10000)
X = v_inverse_cdf(uniform_samples)
X

```

```

[ ]: array([0.01402579, 0.83529294, 0.14197613, ..., 0.70647918, 0.61109145,
          0.89915854], shape=(10000,))

```

0.1.8 2.10

```

"""
2.10. Generate synthetic data using the inverse CDF by transforming uniform
↪ samples. Once you have your inverse CDF, code it up and use it to create
↪ synthetic samples from the last step. To do so, first generate 10,000
↪ samples from the uniform distribution and then feed those uniform variates
↪ through the inverse CDF to generate synthetic variates from our wait time
↪ model. Using those samples, compute the mean and standard deviation. Present
↪ the mean and standard deviation in a table comparing (a) the empirical
↪ values computed from wait_times.csv, (b) your theoretical model values
↪ calculated earlier, and (c) your computed values calculated from your
↪ synthetic model. How do they compare?
"""

synthetic_mean = np.mean(X)
synthetic_std = np.std(X, ddof=1)

# Empirical stats from wait_times.csv
empirical_mean = np.mean(wait_times)
empirical_std = np.std(wait_times, ddof=1)

results = pd.DataFrame({
    "Type": ["Empirical (wait_times.csv)", "Theoretical (from 1.4)", "Synthetic
↪ (Inverse CDF)"],
    "Mean": [empirical_mean, theoretical_mean, synthetic_mean],
    "Std Dev": [empirical_std, theoretical_std, synthetic_std]
})

```

```
results
```

```
[ ]:
      Type      Mean  Std Dev
0 Empirical (wait_times.csv) 0.404196 0.286283
1 Theoretical (from 1.4) 0.100000 0.489900
2 Synthetic (Inverse CDF) 0.417880 0.282384
```

```
# simulated_samples = v_inverse_cdf(uniform_samples)
# # plot histogram of simulated samples
# plt.hist(simulated_samples, bins=30, density=True, alpha=0.6, color='g',
#           label='Simulated Samples Histogram')
# # overlay theoretical pdf
# x_vals = np.linspace(0, 1, 400)
# pdf_vals = v_pdf(x_vals)
# plt.plot(x_vals, pdf_vals, 'r--', label='Theoretical PDF')
# plt.xlabel('x')
# plt.ylabel('Density')
# plt.title('Histogram of Simulated Samples vs Theoretical PDF')
# plt.legend()
# plt.grid(True)
# plt.show()
```

0.1.9 2.11

Collecting scipy

```
Downloading scipy-1.17.0-cp312-cp312-macosx_10_14_x86_64.whl.metadata (62 kB)
Requirement already satisfied: numpy<2.7,>=1.26.4 in
./venv/lib/python3.12/site-packages (from scipy) (2.4.1)
Downloading scipy-1.17.0-cp312-cp312-macosx_10_14_x86_64.whl (31.4 MB)
31.4/31.4 MB
17.0 MB/s 0:00:01m0:00:0100:01
Installing collected packages: scipy
Successfully installed scipy-1.17.0
```

```
from scipy.stats import ks_2samp

# KS Test 1: Empirical vs Synthetic Model
ks_stat_model, pval_model = ks_2samp(wait_times, X)

print("KS Test: Empirical vs Synthetic Model")
print("KS statistic:", ks_stat_model)
print("p-value:", pval_model)

if pval_model > 0.05:
    print("Fail to reject H0: Data are consistent with the model distribution.")
```

```

else:
    print("Reject H0: Data do NOT match the model distribution.")

print()

# KS Test 2: Empirical vs Uniform
ks_stat_uniform, pval_uniform = ks_2samp(wait_times, uniform_samples)

print("KS Test: Empirical vs Uniform")
print("KS statistic:", ks_stat_uniform)
print("p-value:", pval_uniform)

if pval_uniform > 0.05:
    print("Fail to reject H0: Data look uniform (unexpected).")
else:
    print("Reject H0: Data are NOT uniform.")

```

KS Test: Empirical vs Synthetic Model
 KS statistic: 0.0633
 p-value: 0.3960608404537166
 Fail to reject H0: Data are consistent with the model distribution.

KS Test: Empirical vs Uniform
 KS statistic: 0.1704
 p-value: 1.9143182142574203e-05
 Reject H0: Data are NOT uniform.

0.1.10 2.12

```

# X is the synthetic samples I already generated from the inverse CDF
# 1) P(wait > 6 hours) -> 6/8 of day
p_gt_6 = np.mean(X > 6/8)

# 2) P(wait < 1 hour) -> 1/8 of day
p_lt_1 = np.mean(X < 1/8)

# 3) P(wait < 4 | already waited 3)
p_between_3_4 = np.mean((X > 3/8) & (X < 4/8))
p_gt_3 = np.mean(X > 3/8)
p_cond = p_between_3_4 / p_gt_3

# 4) P(3 < wait < 5 hours)
p_between_3_5 = np.mean((X > 3/8) & (X < 5/8))

# 5) 90th percentile
p90 = np.percentile(X, 90)

```

```
# 6) 99th percentile
p99 = np.percentile(X, 99)

print("P(wait > 6 hours):", p_gt_6)
print("P(wait < 1 hour):", p_lt_1)
print("P(wait < 4 | waited 3):", p_cond)
print("P(3 < wait < 5 hours):", p_between_3_5)
print("90th percentile (hours):", p90 * 8)
print("99th percentile (hours):", p99 * 8)
```

```
P(wait > 6 hours): 0.1664
P(wait < 1 hour): 0.1849
P(wait < 4 | waited 3): 0.25624752082506946
P(3 < wait < 5 hours): 0.2383
90th percentile (hours): 6.75996826825175
99th percentile (hours): 7.882599445364464
```


0.2 Exercise 3

0.2.1 3.1

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad c = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

part 3.1

- 1/ $AA = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}$
- 2/ $AA^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 5 & 11 \\ 11 & 25 \end{bmatrix}$
- 3/ $Ab = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- 4/ $Ab^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix}$ inner dimensions 2x2 and 2x2 cannot be multiplied & don't match → invalid
- 5/ $bA = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ inner dimensions 2x1 and 2x2 don't match & cannot be multiplied → invalid
- 6/ $b^T A = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 2 \end{bmatrix}$
- 7/ $bb^T = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix}$ mismatching inner dimensions 2x1, 2x1 → invalid
- 8/ $b^T b = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = 2$
- 9/ $bb^T = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ element wise
- 10/ $A \circ A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \circ \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix}$ ↑
- 11/ $b \circ c = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \circ \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$
- 12/ $b^T b^T = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix}$ 1x2, 1x2, mismatching inner dimensions → invalid
- 13/ $b + c^T = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 & 2 \end{bmatrix}$ 2x1 and 1x2 are different shapes & cannot be added → invalid
- 14/ $A^{-1} b = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \frac{1}{-2} \begin{bmatrix} 4 & -2 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$ $\frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$
- 15/ $b^T Ab = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = 2 - 2 = 0$ part 6 result
- 16/ $b A b^T = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix}$ part 5 result → invalid because use in part 5, bA is invalid

0.2.2 3.2

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

b = np.array([[ -1],
```

```

        [ 1]])

c = np.array([[1],
              [2]])

print("A =\n", A)
print("b =\n", b)
print("c =\n", c)

# 1. A @ A
print("\n1. A @ A =\n", A @ A)

# 2. A @ A.T
print("\n2. A @ A.T =\n", A @ A.T)

# 3. A @ b
print("\n3. A @ b =\n", A @ b)

# 4. A @ b.T (invalid)
# print(A @ b.T) # cannot compute: (2x2) @ (1x2)

# 5. b @ A (invalid)
# print(b @ A) # cannot compute: (2x1) @ (2x2)

# 6. b.T @ A
print("\n6. b.T @ A =\n", b.T @ A)

# 7. b @ b (invalid)
# print(b @ b) # cannot compute: (2x1) @ (2x1)

# 8. b.T @ b
print("\n8. b.T @ b =\n", b.T @ b)

# 9. b @ b.T
print("\n9. b @ b.T =\n", b @ b.T)

# 10. A * A (elementwise)
print("\n10. A * A =\n", A * A)

# 11. b * c (elementwise)
print("\n11. b * c =\n", b * c)

# 12. b.T @ b.T (invalid)
# print(b.T @ b.T) # cannot compute: (1x2) @ (1x2)

# 13. b + c.T (invalid)
# invalid but does not error, produces incorrect output instead

```

```

print(" **** SPECIAL CASE ***** Produces incorrect output instead of error_
↳because Numpy applies broadcasting instead of the valid linear algebra_
↳operation:")
print(b + c.T)      # cannot compute: shapes (2x1) and (1x2)

# 14.  $A^{-1} @ b$ 
A_inv = np.linalg.inv(A)
print("\n14.  $A^{-1} @ b = \backslash n$ ", A_inv @ b)

# 15.  $b.T @ A @ b$ 
print("\n15.  $b.T @ A @ b = \backslash n$ ", b.T @ A @ b)

# 16.  $b @ A @ b.T$  (invalid because  $b @ A$  is invalid)
# print(b @ A @ b.T) # cannot compute: b @ A is invalid

```

```

A =
[[1 2]
 [3 4]]
b =
[[-1]
 [ 1]]
c =
[[1]
 [2]]

1.  $A @ A =$ 
[[ 7 10]
 [15 22]]

2.  $A @ A.T =$ 
[[ 5 11]
 [11 25]]

3.  $A @ b =$ 
[[1]
 [1]]

6.  $b.T @ A =$ 
[[2 2]]

8.  $b.T @ b =$ 
[[2]]

9.  $b @ b.T =$ 
[[ 1 -1]
 [-1  1]]

10.  $A * A =$ 

```

```
[[ 1  4]
 [ 9 16]]
```

11. $b * c =$

```
[[ -1]
 [  2]]
```

**** SPECIAL CASE ***** Produces incorrect output instead of error because Numpy applies broadcasting instead of the valid linear algebra operation:

```
[[0 1]
 [2 3]]
```

14. $A^{-1} @ b =$

```
[[ 3.]
 [-2.]]
```

15. $b.T @ A @ b =$

```
[[0]]
```

0.2.3 3.3

$$\text{l2 norm } \|x\|_2 = \sqrt{\sum x_k^2} = \sqrt{x^T x}$$

$$d_1 = \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix} \quad d_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\|d_1\| = \sqrt{\begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix}} = \sqrt{\frac{1}{2} + \frac{1}{2} + 0} = \sqrt{1} = 1$$

$$\|d_2\| = \sqrt{\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}} = \sqrt{0+0+1} = 1$$

So, $\|d_1\| = 1$, $\|d_2\| = 1$

0.2.4 3.4

Find unit vectors d_3 to form orthonormal basis with d_1, d_2

$$d_3 = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

① Orthogonal to both d_1 and d_2 :

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \cdot \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix} = 0$$

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = 0 \Rightarrow \text{then } z = 0$$

so, $\begin{bmatrix} x \\ y \\ z \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix} = 0$ if $\begin{matrix} 1/\sqrt{2} x - 1/\sqrt{2} y = 0 \\ 1/\sqrt{2} (x - y) = 0 \\ x = y \end{matrix}$

$$\Rightarrow \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ x \\ 0 \end{bmatrix}$$

② Normal (unit length)

$$\|d_3\|_2 = \sqrt{x^2 + x^2 + 0^2} = 1$$

if $\sqrt{2}x^2 = 1 \Rightarrow \sqrt{2}x = 1$
 $x = \pm \frac{1}{\sqrt{2}}$

$$\Rightarrow d_3 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix} \text{ or } \begin{bmatrix} -1/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \end{bmatrix}$$

0.2.5 3.5

```
B = np.array([[1, 2, 3],
              [2, 4, 5],
              [3, 5, 6]])

# calculate the eigenvalues and corresponding eigenvectors of matrix
eigenvalues, eigenvectors = np.linalg.eig(B)
# output results
print("Eigenvalues of B:\n", eigenvalues)
print("\nEigenvectors of B:\n", eigenvectors)
```

Eigenvalues of B:

```
[11.34481428 -0.51572947  0.17091519]
```

Eigenvectors of B:

```
[[-0.32798528 -0.73697623  0.59100905]
 [-0.59100905 -0.32798528 -0.73697623]
 [-0.73697623  0.59100905  0.32798528]]
```

```
# Choose one of the eigenvector/eigenvalue pairs, and and show that lambda * v =
↳ B @ v
lambda_1 = eigenvalues[0]
v_1 = eigenvectors[:, 0]
```

```

print(f"\nVerifying for eigenvalue  = {lambda_1} and corresponding eigenvector v
↳ v = {v_1}:\n")
print(" * v =\n", lambda_1 * v_1)
print("B @ v =\n", B @ v_1)

```

Verifying for eigenvalue = 11.344814282762082 and corresponding eigenvector v
= [-0.32798528 -0.59100905 -0.73697623]:

```

* v =
[-3.72093206 -6.70488789 -8.36085845]
B @ v =
[-3.72093206 -6.70488789 -8.36085845]

```

```

for i in range(eigenvectors.shape[1]):
    for j in range(i + 1, eigenvectors.shape[1]):
        v_i = eigenvectors[:, i]
        v_j = eigenvectors[:, j]
        inner_product = np.dot(v_i, v_j)
        print(f"Inner product of eigenvector {i} and eigenvector {j}:\n
↳ {inner_product:.6f}")

```

```

Inner product of eigenvector 0 and eigenvector 1: -0.000000
Inner product of eigenvector 0 and eigenvector 2: -0.000000
Inner product of eigenvector 1 and eigenvector 2: -0.000000

```

0.3 Exercise 4

Collecting openpyxl

Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)

Collecting et-xmlfile (from openpyxl)

Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)

Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)

Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)

Installing collected packages: et-xmlfile, openpyxl

2/2

[openpyxl]1/2 [openpyxl]

Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5

```
# load in egrid2016.xlsx
import pandas as pd
data = pd.read_excel('HW1/egrid2016.xlsx', header=1)
data.head()
```

```
[ ]:  SEQPLT16 PSTATABB          PNAME          LAT          LON  \
0      1      AK      7-Mile Ridge Wind Project  63.210689 -143.247156
1      2      AK  Agrium Kenai Nitrogen Operations  60.673200 -151.378400
2      3      AK          Alakanuk  62.683300 -164.654400
3      4      AK      Allison Creek Hydro  61.084444 -146.353333
4      5      AK          Ambler  67.087980 -157.856719
```

```
      PLPRMFL  CAPFAC  NAMEPCAP  PLNGENAN  PLCO2EQA
0      WND      NaN      1.8      NaN      NaN
1      NG      NaN      21.6      NaN      NaN
2      DFO  0.05326      2.6  1213.001  1049.863
3      WAT  0.01547      6.5   881.000      0.000
4      DFO  0.13657      1.1  1315.999  1087.881
```

0.3.1 4.1

```
# Which power plant generated the most energy in 2016 (measured in MWh by col_
↳PLNGENAN)?
max_energy_plant = data.loc[data['PLNGENAN'].idxmax()]
print("Power plant that generated the most energy in 2016:", max_energy_plant.
↳PNAME)
print(max_energy_plant)
```

```
Power plant that generated the most energy in 2016: Palo Verde
SEQPLT16      391
PSTATABB      AZ
PNAME      Palo Verde
LAT      33.3881
LON      -112.8617
PLPRMFL      NUC
CAPFAC      0.87801
NAMEPCAP      4209.6
PLNGENAN      32377476.999
PLCO2EQA      0.0
Name: 390, dtype: object
```

0.3.2 4.2

```
# 4.2. Which power plant produced the most CO2 emissions (measured in tons)?
max_co2_plant = data.loc[data['PLCO2EQA'].idxmax()]
print("Power plant that produced the most CO2 emissions in 2016:",
↳max_co2_plant.PNAME)
```

```
print(max_co2_plant)
```

```
Power plant that produced the most CO2 emissions in 2016: James H Miller Jr
SEQPLT16          192
PSTATABB          AL
PNAME      James H Miller Jr
LAT              33.6319
LON             -87.0597
PLPRMFL          SUB
CAPFAC           0.73594
NAMEPCAP          2822.0
PLNGENAN         18193039.0
PLCO2EQA         21724990.489
Name: 191, dtype: object
```

0.3.3 4.3

```
# 4.3. What is the primary fuel of the plant with the most CO2 emissions?
print("Primary fuel of the plant with the most CO2 emissions:", max_co2_plant.
      ↪PLPRMFL)
```

```
Primary fuel of the plant with the most CO2 emissions: SUB
```

0.3.4 4.4

```
# 4.4. What is the name of the northern-most power plant in the United States?

northern_most_plant = data.loc[data['LAT'].idxmax()]
print("Northern-most power plant in the United States:", northern_most_plant.
      ↪PNAME)
print(northern_most_plant)
```

```
Northern-most power plant in the United States: Barrow
SEQPLT16          12
PSTATABB          AK
PNAME      Barrow
LAT              71.292
LON            -156.7786
PLPRMFL          NG
CAPFAC           0.28208
NAMEPCAP          20.3
PLNGENAN         50162.0
PLCO2EQA         44205.17
Name: 11, dtype: object
```


0.3.5 4.5

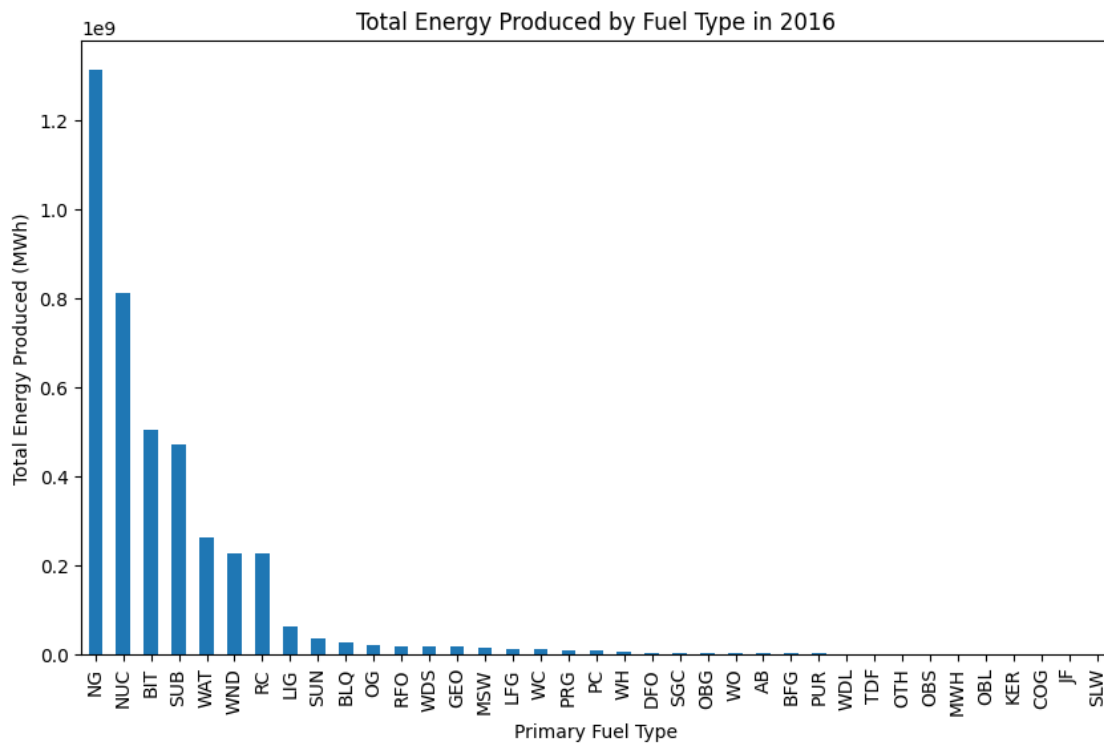
```
# 4.5. What is the state where the northern-most power plant in the United
↪States is located?
print("State of the northern-most power plant in the United States:",
↪northern_most_plant.PSTATABB)
print("Alaska")
```

State of the northern-most power plant in the United States: AK
Alaska

0.3.6 4.6

```
# 4.6. Plot a bar plot showing the amount of energy produced by each fuel type
↪across all plants.

fuel_energy = data.groupby('PLPRMFL')['PLNGENAN'].sum().
↪sort_values(ascending=False)
fuel_energy.plot(kind='bar', figsize=(10,6))
plt.xlabel('Primary Fuel Type')
plt.ylabel('Total Energy Produced (MWh)')
plt.title('Total Energy Produced by Fuel Type in 2016')
plt.show()
```



0.3.7 4.7

From the plot above, NG (natural gas) is the fuel type producing the most energy in the United States.

0.3.8 4.8

```
# 4.8. Which state has the largest number of hydroelectric plants?
hydro_plants = data[data['PLPRMFL'] == 'WAT']
state_hydro_counts = hydro_plants['PSTATABB'].value_counts()
max_hydro_state = state_hydro_counts.idxmax()
max_hydro_count = state_hydro_counts.max()
print(f"State with the largest number of hydroelectric plants:
↳ {max_hydro_state} with {max_hydro_count} plants.")
```

State with the largest number of hydroelectric plants: CA with 264 plants.

0.3.9 4.9

```
# 4.9. Which state has generated the most energy (MWh) using coal? You may also
↳ want to explore the documentation for the isin() method for pandas. Note: in
↳ the eGrid documentation, there are multiple types of coal listed; be sure to
↳ factor in each type of coal.

coal_types = ['BIT', 'LIG', 'SUB', 'RC', 'WC']

coal_plants = data[data['PLPRMFL'].isin(coal_types)]
state_coal_energy = coal_plants.groupby('PSTATABB')['PLNGENAN'].sum()
max_coal_state = state_coal_energy.idxmax()
max_coal_energy_value = state_coal_energy.max()

print(f"State with the most energy generated using coal: {max_coal_state} with
↳ {max_coal_energy_value} MWh.")
```

State with the most energy generated using coal: TX with 122545095.0 MWh.

0.3.10 4.10

```
"""
Which primary fuel produced the most CO2 emissions in the United States? We
↳ would like to compare natural gas, coal, oil, and renewables but the current
↳ categories are much more specific than that. As a first step, group the data
↳ as shown below, replacing the existing labels with the replacements
↳ suggested. For example, BIT and LIG should be replaced with COAL.

COAL = BIT, LIG, RC, SUB, WC
OIL = DFO, JF, KER, RFO, WO
GAS = BFG, COG, LFG, NG, OG, PG, PRG
RENEW = GEO, SUN, WAT, WDL, WDS, WND
```

You may want to create a function that does this replacement prior to running your code. You can check whether or not it was successful by verifying that each of the values that should be replaced has been replaced - check that before moving on with the question.

You will want to use 'PLCO2EQA' to answer this question as it's the quantity of emissions each plant generates.

"""

```
def map_fuel_type(fuel_code):
    if fuel_code in ['BIT', 'LIG', 'RC', 'SUB', 'WC']:
        return 'COAL'
    elif fuel_code in ['DFO', 'JF', 'KER', 'RFO', 'WO']:
        return 'OIL'
    elif fuel_code in ['BFG', 'COG', 'LFG', 'NG', 'OG', 'PG', 'PRG']:
        return 'GAS'
    elif fuel_code in ['GEO', 'SUN', 'WAT', 'WDL', 'WDS', 'WND']:
        return 'RENEW'
    else:
        return 'OTHER'

data['FUEL_GROUP'] = data['PLPRMFL'].apply(map_fuel_type)
fuel_co2_emissions = data.groupby('FUEL_GROUP')['PLCO2EQA'].sum()
max_co2_fuel = fuel_co2_emissions.idxmax()
max_co2_value = fuel_co2_emissions.max()
print(f"Primary fuel that produced the most CO2 emissions: {max_co2_fuel} with {max_co2_value} tons.")
```

Primary fuel that produced the most CO2 emissions: COAL with 1398668540.658 tons.

```
!jupyter nbconvert --to pdf assignment1.out.ipynb --TemplateExporter.
--exclude_input_prompt=True
```

Requirement already satisfied: nbconvert in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (4.14.3)
Requirement already satisfied: bleach!=5.0.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: Jinja2>=3.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in

/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.9.1)
 Requirement already satisfied: jupyterlab-pygments in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.3.0)
 Requirement already satisfied: markupsafe>=2.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.0.3)
 Requirement already satisfied: mistune<4,>=2.0.3 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (3.2.0)
 Requirement already satisfied: nbclient>=0.5.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (0.10.4)
 Requirement already satisfied: nbformat>=5.7 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.10.4)
 Requirement already satisfied: packaging in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (25.0)
 Requirement already satisfied: pandocfilters>=1.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (1.5.1)
 Requirement already satisfied: pygments>=2.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (2.19.2)
 Requirement already satisfied: traitlets>=5.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbconvert) (5.14.3)
 Requirement already satisfied: webencodings in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
 Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
 Requirement already satisfied: platformdirs>=2.5 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-core>=4.7->nbconvert) (4.5.1)
 Requirement already satisfied: jupyter-client>=6.1.12 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from nbclient>=0.5.0->nbconvert) (8.8.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
 Requirement already satisfied: pyzmq>=25.0 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (27.1.0)
 Requirement already satisfied: tornado>=6.4.1 in /Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-

```

packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.4)
Requirement already satisfied: fastjsonschema>=2.15 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from nbformat>=5.7->nbconvert) (4.26.0)
Requirement already satisfied: attrs>=22.2.0 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
Requirement already satisfied: six>=1.5 in /Users/aeshagandhi/Downloads/MIDS-
Sp26/IDS-705/.venv/lib/python3.12/site-packages (from python-
dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
Requirement already satisfied: typing-extensions>=4.4.0 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from referencing>=0.28.4->jsonschema>=2.6->nbformat>=5.7->nbconvert)
(4.15.0)
Requirement already satisfied: soupsieve>=1.6.1 in
/Users/aeshagandhi/Downloads/MIDS-Sp26/IDS-705/.venv/lib/python3.12/site-
packages (from beautifulsoup4->nbconvert) (2.8.3)
Requirement already satisfied: pandoc in /Users/aeshagandhi/Downloads/MIDS-
Sp26/IDS-705/.venv/lib/python3.12/site-packages (2.4)
Requirement already satisfied: plumbum in /Users/aeshagandhi/Downloads/MIDS-
Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandoc) (1.10.0)
Requirement already satisfied: ply in /Users/aeshagandhi/Downloads/MIDS-
Sp26/IDS-705/.venv/lib/python3.12/site-packages (from pandoc) (3.11)
[NbConvertApp] Converting notebook assignment1.out.ipynb to pdf
[NbConvertApp] Support files will be in assignment1.out_files/
[NbConvertApp] Making directory ./assignment1.out_files
[NbConvertApp] Writing 86228 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 2672179 bytes to assignment1.out.pdf

```
